

Rozdział 4

Testy

Aplikacja FareShare została przetestowana zarówno testami automatycznymi, jak i manualnymi, obejmującymi różne aspekty funkcjonalności aplikacji. Przeprowadzone testy mają na celu weryfikację poprawności działania poszczególnych warstw aplikacji oraz ich integracji z resztą systemu.

4.1 Zakres testów

Testy automatyczne obejmowały weryfikację funkcjonalności aplikacji poprzez:

- 149 testów jednostkowych (JUnit 5),
- 91 testów integracyjnych (Spring Boot Test),
- pokrycie kodu na poziomie ok. 80% (według JaCoCo).

Do pokrycia kodu wykorzystano narzędzie JaCoCo (Java Code Coverage) [3], które dostarcza szczegółowych informacji na temat wykonanych linii kodu, metod i klas podczas testów automatycznych.

backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
share.fare.backend.config		37%		2%	29	46	127	185	8	25	2	7
share.fare.backend.exception		63%		n/a	14	49	28	101	14	49	4	24
share.fare.backend.mapper		79%		55%	47	72	48	225	16	36	0	11
share.fare.backend.controller		75%		n/a	16	56	24	101	16	56	5	13
share.fare.backend.service		96%		79%	27	194	11	652	5	135	0	16
share.fare.backend.entity		81%		50%	12	25	8	56	6	19	1	11
share.fare.backend.controller.groupchat		45%		n/a	1	3	11	15	1	3	1	2
share.fare.backend		37%		n/a	1	2	2	3	1	2	0	1
share.fare.backend.service.strategy		100%		100%	0	18	0	49	0	11	0	6
share.fare.backend.util		100%		n/a	0	3	0	18	0	3	0	2
Total	1,092 of 6,025	81%	103 of 254	59%	147	468	259	1,405	67	339	13	93

Rysunek 4.1: Ogólne statystyki pokrycia kodu według JaCoCo

share.fare.backend.service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
VoteService		93%		75%	4	15	2	71	2	11	0	1
ActivityService		83%		16%	4	10	2	23	1	7	0	1
SecurityService		76%		66%	2	8	2	13	0	5	0	1
ExpenseService		96%		83%	4	33	2	103	0	20	0	1
GroupBalanceService		97%		81%	4	15	0	50	1	7	0	1
GroupInvitationService		98%		91%	1	16	1	77	0	10	0	1
SettlementService		98%		85%	3	19	0	72	1	12	0	1
FriendInvitationService		98%		83%	2	15	1	67	0	9	0	1
UserInfoService		97%		50%	1	7	1	22	0	6	0	1
GroupMembershipService		100%		100%	0	15	0	40	0	13	0	1
GroupService		100%		83%	1	15	0	34	0	12	0	1
AuthenticationService		100%		100%	0	4	0	30	0	3	0	1
JwtService		100%		75%	1	12	0	27	0	10	0	1
UserService		100%		n/a	0	6	0	10	0	6	0	1
FriendshipService		100%		n/a	0	2	0	8	0	2	0	1
NotificationService		100%		n/a	0	2	0	5	0	2	0	1
Total	95 of 2,938	96%	24 of 116	79%	27	194	11	652	5	135	0	16

Rysunek 4.2: Pokrycie kodu w warstwie serwisowej

share.fare.backend.controller

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ExpenseController		0%		n/a	4	4	8	8	4	4	1	1
SettlementController		0%		n/a	4	4	8	8	4	4	1	1
UserInfoController		0%		n/a	4	4	4	4	4	4	1	1
GroupBalanceController		0%		n/a	2	2	2	2	2	2	1	1
AuthenticationController		0%		n/a	2	2	2	2	2	2	1	1
GroupController		100%		n/a	0	7	0	14	0	7	0	1
GroupInvitationController		100%		n/a	0	6	0	13	0	6	0	1
FriendInvitationController		100%		n/a	0	6	0	13	0	6	0	1
UserController		100%		n/a	0	7	0	9	0	7	0	1
ActivityController		100%		n/a	0	4	0	8	0	4	0	1
VoteController		100%		n/a	0	4	0	8	0	4	0	1
GroupMembershipController		100%		n/a	0	4	0	8	0	4	0	1
FriendshipController		100%		n/a	0	2	0	4	0	2	0	1
Total	124 of 496	75%	0 of 0	n/a	16	56	24	101	16	56	5	13

Rysunek 4.3: Pokrycie kodu w warstwie kontrolerów

Testy manualne obejmowały:

- przetestowanie 57 endpointów API w aplikacji Postman,
- weryfikację odpowiedniego zabezpieczenia endpointów,
- sprawdzanie poprawności przetwarzania danych przesyłanych w treści żądania (body), parametrach (query params) oraz zmiennych ścieżki (path variables).

4.2 Kluczowe przypadki testowe

W procesie tworzenia testów wyznaczono poniższe przypadki testowe:

Testy jednostkowe

- **Zarządzanie atrakcjami:** Weryfikacja tworzenia/edycji/usuwania atrakcji (7 przypadków)

- **Autentykacja:** Testy rejestracji, logowania, walidacji tokenów (5 przypadków)
- **Wydatki:** Podział kosztów, aktualizacja sald (17 przypadków)
- **Zaproszenia do znajomych:** Zapraszanie do znajomych, akceptowanie i odrzucanie zaproszeń (19 przypadków)
- **Zarządzanie znajomościami:** Pobieranie listy znajomych, usuwanie znajomych (4 przypadki)
- **Grupy:** Tworzenie, edycja, usuwanie grup (12 przypadków)
- **Zaproszenia do grup:** Zapraszanie do grup, akceptowanie i odrzucanie zaproszeń (20 przypadków)
- **Token JWT:** Generowanie, walidacja oraz odczyt danych z tokena (6 przypadków)
- **Rozliczanie wydatków:** Podział wydatków, aktualizacja sald (15 przypadków)
- **Balans wydatków Grupy:** Pobieranie balansu grupy, pobieranie listy minimalnych transakcji (6 przypadków)
- **Dane użytkownika:** Pobieranie, edytowanie danych użytkownika (7 przypadków)
- **Konto użytkownika:** Pobieranie użytkowników, edycja i usuwanie kont (6 przypadków)
- **Głosowanie nad atrakcjami:** Dodawanie, edytowanie, usuwanie głosów (18 przypadków)
- **Strategie podziału wydatków:** Weryfikacja strategii podziału wydatków (4 przypadki)
- **Inne moduły:** Klasy pomocnicze (3 przypadki)

Testy integracyjne

- **API atrakcji:** Tworzenie, edytowanie, usuwanie atrakcji (9 przypadków)
- **API zapraszania do znajomych:** Wysyłanie, akceptowanie, odrzucanie zaproszeń do znajomych (10 przypadków)
- **API zarządzania znajomościami:** Pobieranie listy znajomych, usuwanie znajomych (4 przypadki)
- **API grup:** Tworzenie, edytowanie, usuwanie grup (14 przypadków)
- **API czatu grupowego:** Wysyłanie wiadomości (3 przypadki)
- **API zaproszeń do grup:** Wysyłanie, akceptowanie, odrzucanie zaproszeń do grup (12 przypadków)
- **API członkostwa grupy:** Dodawanie, pobieranie, usuwanie członków grupy (11 przypadków)

- **API użytkownika:** Pobieranie, edytowanie danych użytkownika (20 przypadków)
- **API systemu głosowania:** Dodawanie, edytowanie, usuwanie głosów (8 przypadków)

Tabela 4.1: Przykładowe przypadki testowe

ID	Opis	Wynik
PT1	Utworzenie atrakcji z prawidłowymi danymi	Sukces (201)
PT2	Logowanie nieistniejącego użytkownika	Błąd (404)
IT3	Usunięcie grupy z nieautoryzowanym dostępem	Błąd (403)
IT4	Podział wydatków z procentową strategią podziału	Poprawny podział kwoty

4.3 Podsumowanie wyników testów automatycznych

Tabela 4.2: Statystyki testów jednostkowych

Moduł	Liczba testów	Stan
Zarządzanie atrakcjami	7	100% sukces
Autentykacja	5	100% sukces
Zarządzanie wydatkami	17	100% sukces
Zarządzanie zaproszeniami do znajomych	19	100% sukces
Serwis zarządzania znajomościami	4	100% sukces
Zarządzanie grupami	12	100% sukces
Zarządzanie balansem grupy	6	100% sukces
Zarządzanie zaproszeniami do grupy	20	100% sukces
Zarządzanie tokenem JWT	6	100% sukces
Zarządzanie rozliczeniami wydatków	15	100% sukces
Zarządzanie danymi użytkownika	7	100% sukces
Zarządzanie kontem użytkownika	6	100% sukces
Serwis głosowania nad atrakcjami	18	100% sukces
Serwis doboru strategii podziału wydatków	4	100% sukces
Inne moduły	3	100% sukces

Tabela 4.3: Statystyki testów integracyjnych

Moduł	Liczba testów	Stan
API atrakcji	9	100% sukces
API zaproszeń do znajomych	10	100% sukces
API zarządzania znajomościami	4	100% sukces
API grupy	14	100% sukces
API czatu grupowego	3	100% sukces
API zaproszeń do grupy	12	100% sukces
API członkostwa grupy	11	100% sukces
API użytkownika	20	100% sukces
API systemu głosowania	8	100% sukces

Wnioski

Wszystkie testy zakończyły się powodzeniem, a system odpowiednio sobie radzi z przypadkami brzegowymi. Aplikacja jest prawidłowo zabezpieczona przed potencjonalnymi atakami, a dane użytkowników są chronione. Dzięki licznym testom jednostkowym rozwój aplikacji został znacząco ułatwiony, ponieważ umożliwił wczesne wykrywanie błędów. Testy integracyjne pozwoliły na weryfikację poprawnych interakcji między komponentami systemu. Z obecną ilością testów uzyskano pokrycie kodu na poziomie 80% (według JaCoCo), co stanowi zadowalający wynik.